Implementing Signal Protocol

Cryptography CS 411 & CS 507 Term Project for Fall 2025

Erkay Savaş & Atıl Utku Ay
Computer Science & Engineering
Sabancı University
İstanbul

Abstract

You are required to develop a simplified version of the Signal Protocol, which provides forward secrecy and deniability. Working on this project will provide you with insight into a practical cryptographic protocol, a variant of which is used in various applications such as WhatsApp.

1 Introduction

The project has three phases:

- **Phase I:** Developing software for the Public Key Registration
- **Phase II:** Developing software for communicating with other clients
- **Phase III:** Developing software for conference keying and communicating with other clients using PQXDH

1. All coding development will be in the Python programming language. More information about the project is given in the subsequent sections.
2. You should connect to the university's VPN to be able to connect to the server **if you are connecting from outside of campus**. Otherwise, you won't be able to send your requests. Check the IT's website (`https://mysu.sabanciuniv.edu/it/en/remote-vpn-access`)

2 Phase I: Developing software for the Public Key Registration

In this phase of the project, you are required to upload one file: "Client_Phase1.py". You will be provided with "Client_Phase1_Basics.py", which includes all required communication codes.

In this protocol, Elliptic Curve Cryptography (ECC) is used in the key exchange protocol and digital signature algorithm with the Ed25519 curve. You should select "`Ed25519`" for the curve in your Python code. There are three types of public keys; Identity Key (`IK`), Signed Pre-key (`SPK`) and One-time Pre-key (`OTK`).

2.1 Identity Key (IK)

Identity Key (IK) consists of a long-term public-private key pair, which each party generates once and uses to sign his/her SPK as shown in Section 2.2.

2.1.1 Registration of Identity Keys

The identity public key of the server $\texttt{IK}_S\texttt{.Pub}$ is given below.

```
X:0x2eef2e2656fb3d8c3c4932c679fbca121c2ea5fe26deecd800bc9311ef06f06
Y:0x17a8a7f452068d1157a974abc69cd5ae83d528936b4c1d8dab6095d28eeedcc0
```

Firstly, you are required to generate an identity private and public key pair $\texttt{IK}_A\texttt{.Pri}$ and $\texttt{IK}_A\texttt{.Pub}$ for yourself. The key generation is described in the "Key Generation" algorithm in Section 2.4. Then, you are required to register your public key with the server. The identity key registration operation consists of four steps:

1. After you generate your identity key pair, you should sign your ID (e.g., 26045). The details of the signature scheme are given in the "Signature Generation" algorithm in Section 2.4. Then, you will send a message, which contains your student ID, the signature tuple, and $\texttt{IK}_A\texttt{.Pub}$, to the server. The message format is

   ```
   {'ID': stuID, 'R.X': R.x, 'R.Y': R.y, 'S': s, 'IKPUB.X': ikpub.x, 'IKPUB.Y': ikpub.y}
   ```

   where stuID is your student ID; R.x and R.y are the x and y coordinates of the point R, which is the point component of the signature; s is the scalar component of the signature; and ikpub.x and ikpub.y are the x and y coordinates of $\texttt{IK}_A\texttt{.Pub}$, respectively.

2. If your message is verified by the server successfully, you will receive an e-mail, which includes your ID and a 6-digit verification code: code.

3. After receiving the verification e-mail, you will send another message to the server to authenticate yourself. The message format is "{'ID': stuID, 'CODE': code}", where code is the verification code which, you have received in the previous step. A sample message is given below.

   ```
   {'ID': 26045, 'CODE': 209682}
   ```

4. If you send the correct verification code, you will receive an acknowledgement message via e-mail, which states that you are registered with the server successfully and contains a code to reset your identity key if you need. (You must save the reset code to delete your IK from the server, in case you need to. (e.g., your identity key is lost or compromised))
   Note: Once you receive an acknowledgment email, your Identity key will be saved locally on your computer.

2.1.2 Resetting your IK

If you lose your private identity key $\texttt{IK}_A\texttt{.Pri}$, and need to reset your identity key pair, you should send a message to the server to delete your public identity key $\texttt{IK}_A\texttt{.Pub}$ from the server. The message format is "{'ID': stuID, 'RCODE': rcode}", where rcode is reset code which was provided in the acknowledgement e-mail. A sample message is as follows

```
{'ID': 26045, 'RCODE': 945333}
```

If you lose the reset code, you should send an e-mail to cs411tpserver@sabanciuniv.edu. Your IK will be deleted from the server after 8 hours.

2.2 Signed Pre-key (SPK)

Signed Pre-key is another long-term key pair, which all parties generate once and is used in the registration of one-time pre-keys (OTK).

2.2.1 Registration of SPK

After you have registered your identity key, you are required to generate one pair of signed pre-key; $\texttt{SPK}_A\texttt{.Pub}$ and $\texttt{SPK}_A\texttt{.Pri}$. Then, you must sign the public key part of the signed pre-key, $\texttt{SPK}_A\texttt{.Pub}$ using your identity key $\texttt{IK}_A$. The signature, for which a scheme is given in Section 2.4, must be generated for the concatenated form of the public signed pre-key: ($\texttt{SPK}_A\texttt{.Pub.x} \parallel \texttt{SPK}_A\texttt{.Pub.y}$). Finally, you must send your signed pre-key to the server in the form of

```
{'ID': stuID , 'SPKPUB.X': spkpub.x, 'SPKPUB.Y': spkpub.y, 'R.X': R.x, 'R.Y': R.y,
'S': s},
```

where R.x, R.y and s denote the signature tuple.

2.2.2 Resetting your SPK

If you lose your private signed pre-key $\texttt{SPK}_A\texttt{.Pri}$, and need to reset your signed pre-key pair, you should sign your stuID using your identity key $\texttt{IK}_A$ and send a message to the server to delete your public identity key $\texttt{SPK}_A\texttt{.Pub}$ from the server. The message format is

```
{'ID': stuID, 'R.X': R.x, 'R.Y': R.y, 'S': s}
```

2.3 One-time Pre-key (OTK)

One-time Pre-keys are the keys which are used to generate symmetric session keys in communication with other clients. Therefore, each client in the system must register his/her OTKs to the server before the communication.

2.3.1 Generating HMAC Key ($\texttt{K}_{\texttt{HMAC}}$)

In the registration of OTKs, a hash-based MAC (HMAC) function will be used for authentication to provide deniability (instead of digital signature). Therefore, you should generate a symmetric

HMAC Key ($\mathtt{K_{HMAC}}$) before the registration of `OTK`s. $\mathtt{K_{HMAC}}$ will be computed as follows:

- `T = SPK`$_A$`.Pri` $\cdot$ `IK`$_S$`.Pub` (Diffie-Hellman with `SPK` of the client and the `IK` of the server)
- `U` $= \{$`b'TheHMACKeyToSuccess'` $\|$ `T.y` $\|$ `T.x`$\}$
- $K_{\mathtt{HMAC}} = \mathtt{SHA3_256}(\mathrm{U})$

2.3.2 Registration of `OTK`s

Before communicating with other clients, you must generate 10 one-time public and private key pairs, namely $\mathtt{OTK}_{A_0}, \mathtt{OTK}_{A_1}, \ldots, \mathtt{OTK}_{A_9}$. The key generation is described in the "Key Generation" algorithm in Section 2.4.

Then, you must compute an HMAC value for each of your public one-time pre-key $\mathtt{OTK_{A_i}}$ using the `HMAC-SHA256` function and $\mathtt{K_{HMAC}}$ as the key. The HMAC value must be generated for concatenated form of the one-time public keys (e.g., ($\mathtt{OTK}_{A_i}$`.Pub.x` $\|$ $\mathtt{OTK}_{A_i}$`.Pub.y`) for the ith one-time pre-key). Finally, you must send your one-time public keys to the server in the form of

```
{'ID': stuID, 'KEYID':i , 'OTKI.X': OTKi.x, 'OTKI.Y': OTKi.y, 'HMACI': hmaci},
```

where `i` is the ID of your one-time pre-key. You must start generating your one-time pre-keys with IDs from 0 and follow the order.

2.3.3 Resetting your `OTK`s

If you lose the private part of your one-time pre-keys, and need to reset your one-time pre-key pairs, you should sign your `stuID` using your identity key `IK` and send a message to the server to delete your registered one-time pre-keys. The message format is

```
{'ID': stuID, 'R.X': R.x, 'R.Y': R.y, 'S': s}
```

2.4 The Digital Signature Scheme

Here, you will develop a Python code that includes functions for signing any given message and verifying the signature. For this digital signature scheme, you will use an algorithm, which consists of three functions as follows (note that || represent the byte-concatenation operation):

- **Key Generation:** A user picks a random secret key $0 < s_A < n - 1$ and computes the public key $Q_A = s_A P$.
- **Signature Generation:** Let m be an arbitrary length message. The signature is computed as follows:
  1. $h_1 = SHA3_512(s_A)$
  2. $r = SHA3_512(h_1[32:]||m)$ where $h[32:]$ denotes the upper 32 bytes (second half) of h_1.
  3. $R = r \cdot P$
  4. $h_2 = SHA3_512(R.x||R.y||Q_A.x||Q_A.y||m)$

5. $s = (r + s_A \cdot h_2) \pmod n$
6. The signature for m is the tuple (R, s).

- **Signature Verification:** Let m be a message and the tuple (R, s) is a signature for m. The verification proceeds as follows:

  1. $h = SHA3_512(R.x||R.y||Q_A.x||Q_A.y||m)$
  2. $v_1 = s \cdot P$
  3. $v_2 = R + (h \cdot QA)$
  4. Accept the signature only if $v_1 = v_2$.
  5. Reject it otherwise.

IMPORTANT:

- This step is about generating the necessary keys for our communication protocol. Therefore, make sure to run each step alone. For instance, if you already generated your `Identity Key`, then you moved to the part where you generate the `Signed Pre-Key`, you make sure you do not run the previous code of generating `IK` again.

- **WHAT CAN GO WRONG?** The reason behind the previous note is that you may end up asking the server to send you the `rcode` multiple times (of course after resetting your IK). After a while, you may not see the message (falling within the SPAM box). Anyhow, you can test all the functions at once if you have completed the respective phase and want to see if everything is working flawlessly.

3 Phase II: Developing software to communicate with other clients

In this phase of the project, you are required to upload one file: "Client_Phase2.py". You will be provided with "Client_Phase2_Basics.py", which includes all the necessary communication codes. Moreover, you will find sample outputs for this phase in 'sample_run_2.txt', which is also provided on SUCourse.

This phase of the project is divided into two main parts. In the first part of this phase, you are required to develop a software for downloading five (5) messages from the server, which were originally uploaded to the server by a pseudo-client, which is implemented by us[1].

For the second part of the phase, you will develop software to send messages to other users. (You are not limited to sending messages to your groupmate or pseudo-clients. You may send messages to anybody who takes CS411/507.) In this part, you will also manage your `OTK`s. The details are given in the subsequent sections.

For testing purposes, you may send a message to your groupmate, and s/he may receive your message if you are working in groups of two. Note that if you have not registered with the server yet, you must register. The details for the registration are given in Section 2. Moreover, you must be able to receive messages from other users. If you are working alone, you may communicate with `18007` (the ID of the pseudo-client).

[1] This is indeed an asynchronous messaging application, whereby other users can send you a message even if you are not online. Yes, exactly like the WhatsApp application.

In this protocol, a session key, $\mathtt{K_S}$, is generated (the details will be explained in Section 3.1.1) for each message block[2] using X3DH. Then, one encryption and one HMAC key are generated from $\mathtt{K_S}$ for each message in the message block using a key derivation function `KDF` chain (the details will be explained in Section 3.1.2). The messages are encrypted using AES256-CTR and authentication is provided using HMAC-SHA256[3].

3.1 End-to-End Secure Communication Scheme

The protocol, which you implement in this project, provides end-to-end security in communication. In other words, no other party (including the server) can read and/or modify original messages. Moreover, forward secrecy is another feature of the protocol. Therefore, long-term keys should not be used for encryption. In order to achieve the end-to-end secure communication with forward secrecy, the communicating parties must use one-time keys to compute a session key ($\mathtt{K_S}$). Then, they must generate an encryption and an HMAC key for each message of the corresponding message block using a key derivation function (`KDF`) chain. Finally, they encrypt the messages using AES256-CTR for confidentiality and compute the HMAC-SHA256 value of the ciphertext for integrity.

3.1.1 Session Key ($\mathtt{K_S}$)

As mentioned before, both parties should establish a session key ($\mathtt{K_S}$) for a message block to generate encryption and HMAC keys at first. $\mathtt{K_S}$ is computed using X3DH (Extended Triple Diffie-Hellman) key exchange protocol. Therefore, when a user wants to send a message block to another user, s/he requests the *pre-key bundle* of the receiver from the server. The pre-key bundle contains the public `IK`, public `SPK`, and one of the `OTK`s of the receiver. Then, the sender generates a DH key pair, which is called "Ephemeral Key (`EK`)" and it is used in the DH key exchange along with the sender's public `IK`, and receiver's `IK`, `SPK`, and `OTK`. The computation of ($\mathtt{K_S}$) in the receiver side is given below:

- $\mathtt{T1} = \mathtt{SPK_A.Pri} \cdot \mathtt{IK_B.Pub}$[4]
- $\mathtt{T2} = \mathtt{IK_A.Pri} \cdot \mathtt{EK_B.Pub}$
- $\mathtt{T3} = \mathtt{SPK_A.Pri} \cdot \mathtt{EK_B.Pub}$
- $\mathtt{T4} = \mathtt{OTK_A.Pri} \cdot \mathtt{EK_B.Pub}$
- $\mathtt{U} = \{\mathtt{T1.x} \parallel \mathtt{T1.y} \parallel \mathtt{T2.x} \parallel \mathtt{T2.y} \parallel \mathtt{T3.x} \parallel \mathtt{T3.y} \parallel \mathtt{T4.x} \parallel \mathtt{T4.y} \parallel \mathtt{b'WhatsUpDoc'}\}$ [5]
- $\mathtt{K_S} = \mathtt{SHA3_256}(\mathtt{U})$

where `A` refers to the receiver and `B` refers to the sender. For example, $\mathtt{EK_B.Pub}$ is public ephemeral key of the sender, and $\mathtt{OTK_A.Pri}$ is the private `OTK` of the receiver.

[2] Message block refer to a group of messages, which are sent by a client consecutively without receiving any message in between from the other party. You can think of WhatsApp as an example. You may send 3 messages consecutively to your friend. Then, s/he may send 2 messages. Finally, you may send 4 messages. And so on. Each of them is called a message block.

[3] Note that, SHA3_256 is used for key generation. On the other hand, SHA2_256 is used for MAC computation.

[4] The sender will compute $\mathtt{T1} = \mathtt{IK_B.Pri} \cdot \mathtt{SPK_A.Pub}$. If you are confused, check how the Diffie-Hellman Key Exchange Protocol is working. Except `T1, T2, T3, and T4`, the rest of the computation is the same for the sender.

[5] `https://youtu.be/QBcK63h3Avw?si=sDh4_iqceebeYVkj`

3.1.2 Key Derivation Function (KDF) Chain

The Key Derivation Function (KDF) is a cryptographic function that takes a secret KDF key (K_{KDF}) and returns an encryption key, an HMAC key, and KDF key for the next iteration. If there is more than one message in a message block, KDF is used more than once to generate different encryption and HMAC keys for each message. The outputs of KDF are computed as follows:

- K_{ENC} = SHA3_256(K_{KDF} || b'JustKeepSwimming')[6]
- K_{HMAC} = SHA3_256(K_{KDF} || K_{ENC} || b'HakunaMatata')
- $K_{KDF.Next}$ = SHA3_256(K_{ENC} || K_{HMAC} || b'OhanaMeansFamily')

After K_S is computed, it will be used as the KDF key for the first message. If there is only one message in the message block, K_{ENC} and K_{HMAC} will be used for encryption and HMAC values, respectively. However, $K_{KDF.Next}$ will not be used (only if there is one message in the block). Otherwise, $K_{KDF.Next}$ is used as the KDF key to generate K_{ENC} and K_{HMAC} for the next message of the message block. Figure 1 shows how the KDF chain is working for a message block which contains 3 messages, m_1, m_2 and m_3 namely[7].

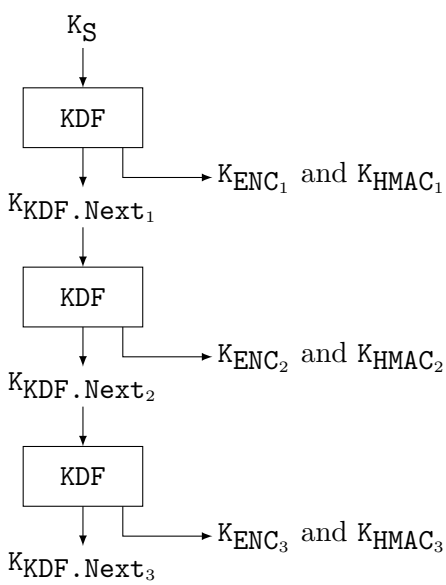

Figure 1: KDF Chain for a message block with 3 messages

In this example, K_S is computed by both parties before the communication and used as K_{KDF} to generate K_{ENC_1} and K_{HMAC_1}, where K_{ENC_1} and K_{HMAC_1} are the encryption and HMAC keys of m_1, respectively. K_{KDF_1} is used to generate K_{ENC_2} and K_{HMAC_2} for m_2. And so on...

3.2 Downloading Messages from the Server

As mentioned above, you will download 5 messages from the server. In order to download one message from the server, you must sign your ID using your Identity Key and send a message to the

[6] https://youtu.be/0Hkn-LSh7es?si=Nfw2oCBrS8x0x_KF

[7] For convenience in implementation, message index starts from 1 instead of 0.

server in the form of

```
{'ID': stuID, 'R.X': R.x, 'R.Y': R.y, 'S': s}
```

The message will be downloaded in the form of

```
{'IDB': stuIDB, 'OTKID': otkID, 'MSGID': msgID, 'MSG': msg , 'IK.X': IK.x ,'IK.Y':
IK.y, 'EK.X': EK.x ,'EK.Y': EK.y },
```

where `stuIDB` is the ID of the sender, `otkID` is the ID of your one-time prekey, which is used to generate session key, `msgID` indicates the order number of the message in the corresponding message block, `msg` contains the nonce, the ciphertext and MAC value of the ciphertext[8], and `EK.x` and `EK.y` are x and y coordinates of the ephemeral public key (`EK.Pub`) of the sender, respectively. Similarly, `IK.x` and `IK.y` are x and y coordinates of the public identity key (`IK.Pub`) of the sender (**Note:** there is only one message format. Therefore, all messages in the same message block contain the same `otkID` and $\mathtt{EK_B}$`.Pub` information.)

3.3 Decrypting the Messages From the Pseudo-Client

In this phase, you will request and receive **five** messages from a pseudo-client, which simulates your correspondence sending messages to you. Therefore, you need to request to generate 5 messages from pseudo-client (via the server) at first. The message format is

```
{'ID': stuID, 'R.X': R.x, 'R.Y': R.y, 'S': s}
```

When you request five messages, the messages will be ready on the server and you may download them (explained in Section 3.2). After you download the messages, you need to compute $\mathtt{K_S}$ and `KDF` chain at first. Then, you should check the MAC values of the messages and decrypt them (4 messages will have valid MAC values, but one of them will not.). Finally you should send a message for each received message, which you downloaded, in the form of:

```
{'IDA': stuIDA, 'IDB': stuIDB, 'MSGID': msgID, 'DECMSG': decmsg },
```

where `stuIDA` and `stuIDB` are the ID of you and the pseudo-client, respectively. `msgID` is the ID of the message (The order of the sent messages will be the same as the received messaged. In other words, you should put the same message ID with the received message.) and `decmsg` is the decrypted form of the message if the MAC value is valid. Otherwise, you should set `decmsg` field to `"INVALIDHMAC"`.

3.4 Displaying the Final Message Block

After receiving the messages from the pseudo-client and **decrypting them successfully**, you are asked to ask the server about whether the pseudo-client has deleted some messages from the received block or not. This case is similar to the case in WhatsApp messages, a sender may delete

[8] `msg = nonce‖ciphertext‖MAC`
Where the nonce is the AES-CTR nonce of 8 bytes. You can conclude the size of the MAC value from the used hash function algorithm.

some messages after sending them. You won't implement the exact scenario. However, you will display the messages of the received block as given in the sample run. Here are the steps:

1. Send the server a request that includes your `stuID` and its signature using your `IK` in the form:
   {'ID': stuID, 'R.X': R.x, 'R.Y': R.y, 'S': s}
   It is the same request as in Step 3.3.

2. After the server verifies your signature, it will send (as a result of your request) a list that has the message IDs (`MSGID`) of the ones that got deleted.

3. Finally, you will display the messages in the same manner as given in the sample run.

4. **Note:** Bear in mind that there is an invalid message among the block. You should not print it as you could not verify its signature. However, the pseudo-client can delete it, and you may receive its `MSGID` as well.

3.5 Sending Messages

In this part of Phase II, you are required to develop software to send messages to other users. Firstly, you are required to generate a session key K_S. The details of session key generation are given in Section 3.1.1. To generate K_S to send a message to another user, you need the receiver's pre-key bundle, which consists of the receiver's identity key, signed pre-key, and one one-time pre-key. Therefore, you must request the pre-key bundle of the receiver from the server. The message format is given below.

{'IDA': stuIDA, 'IDB': stuIDB, 'R.X': R.x, 'R.Y': R.y, 'S': s}

where `stuIDA` and `stuIDB` are your and the receiver's IDs, respectively. `R` and `s` is a signature tuple which is generated for `stuIDB` using your long-term private key. If your request is valid, the server will send the pre-key bundle of the receiver as follows:

{'KEYID': i, 'IK.X': IK.x, 'IK.Y': IK.y, 'SPK.X': SPK.x, 'SPK.Y': SPK.y, 'R.X': R.x, 'R.Y': R.y, 'S': s, 'OTK.X': OTK.x, 'OTK.Y': OTK.y}

where `i` denotes the `OTK` ID of the receiver (you will use it in your message to identify which `OTK` of the receiver you are using). Moreover, `OTK.x` and `OTK.y` are x and y coordinates of the $\mathtt{i}^{\mathtt{th}}$ `OTK` of the receiver. Note that the signature `(R, s)` is generated for the concatenated form of the public signed pre-key (`SPK.Pub.x` $\|$ `SPK.Pub.y`).

Upon receiving the pre-key bundle, you need to verify the pre-key signature. If verification succeeds, you should generate the ephemeral key pair `EK` and compute K_S.

After you computed the K_S, you are required to create a `KDF` chain (Explained in Section 3.1.2). Then, you should encrypt your message/s and compute MAC values using the encryption and MAC key/s. Finally, you should send your message to the server in the form of

{'IDA': stuIDA, 'IDB': stuIDB, 'OTKID': otkID, 'MSGID': msgID, 'MSG': msg, 'IK.X': IK.x ,'IK.Y': IK.y, 'EK.X': EK.x ,'EK.Y': EK.y}

where `stuIDA` and `stuIDB` are the IDs of you and the receiver, respectively. `otkID` denotes the `OTK` ID of the receiver, which is sent to you by the server. `msgID` indicates the message index in the corresponding message block. `msg` includes the nonce, ciphertext and MAC of the ciphertext. `EK.x` and `EK.y` are x and y coordinates of your `EK`. `IKPUB.x` and `IKPUB.y` are x and y coordinates of your public `IK`. (**Note:** If there is more than one message in a message block, you should use the same `otkID` and `EK` information in all messages of the same message block.)

3.6 Status Control

As indicated in several places in previous sections, the protocol, which you are implementing in this project, is an offline protocol. In other words, the messages which are sent by the sender are stored in the server until the receiver requests to retrieve his/her messages. Moreover, `OTK`s, which are generated by the users, are used once (for only one message block) to generate a new session key (K_S). Therefore, you are required to check the server for how many new messages you have and how many `OTK`s are remaining. Moreover, you must generate new `OTK`s and register with the server if needed. To be informed of new messages and remaining `OTK`s, you must send your ID to the server as follows:

```
{'ID': stuID, 'R.X': R.x, 'R.Y': R.y, 'S': s }
```

where `R` and `s` is a signature tuple, which is generated for `stuID` using your long-term private key. If your request is valid, the server will send the number of new messages and remaining `OTK`s. Then, you may either get your messages from the server or register your new `OTK`s with the server.

3.6.1 Registration of new `OTK`s

In Phase I, you have generated 10 `OTK`s and registered with the server, and each `OTK` is used for one message block. On the other hand, you may receive more than 10 message blocks. Therefore, if some of your `OTK`s are used, you must generate new `OTK`s to reach a total of 10 keys, then register them in the server again. The registration protocol is explained in section 2.3.2 As mentioned in section 2.3.2, you must follow the order for key IDs.

You may receive a message from the server saying that you ran out of OTKs. In that case, you will have to re-generate new OTKs and demonstrate that in your output.

3.7 Sending messages to pseudo-clients

This task you should perform is similar to the task in Section 3.3. In Section 3.3, you have received 5 messages from a pseudo-client, decrypted them, then checked which message(s) was(were) deleted. In this phase, you will also re-encrypt [9] the messages you received and send them back to the pseudo-client whose ID number is `18007`.

3.7.1 The process flow

You should download 5 messages from the server and decrypt them. Therefore, you should complete the task in Section 3.3 before this task. Then, you are asked to create a session with the pseudo-client. Later on, you should encrypt the undeleted messages you received with valid MAC values.

[9] Check the API of AES under PyCryptodome for more details about encryption & decryption: https://pycryptodome.readthedocs.io/en/latest/src/cipher/aes.html

Finally, you will send the encrypted messages back to the pseudo-client (i.e., ID: `18007`) in the same order. The details are explained in Section 3.1 and Section 3.5. Note that, for some message blocks, all messages may have been deleted by the pseudo-client; in this case, there will be no messages to send back to the pseudo-client. To test the correctness of your message-sending mechanism, we may ask you to retrieve an additional message block from the pseudo-client during the demonstration.

4 Phase III: Developing software for conference keying & communicating with other clients using PQXDH

In this phase of the project, you are required to upload one file: "Client_Phase3.py". You will be provided with "Client_Phase3_Basics.py", which includes all the necessary communication codes. Moreover, you will find sample outputs for this phase in 'sample_run_3.txt', which is also provided on SUCourse.

This phase of the project is divided into two main parts. In the first part of this phase, you are expected to generate a conference key with 3 other users for group communication. In the second part of this phase, you are expected to implement the communication protocol you implemented in Phase II using Post-Quantum Extended Diffie–Hellman (PQXDH) instead of X3DH.

Please note that for Phase III, you are required to connect to a different port on `harpoon1.sabanciuniv.edu`. In the previous phases of the project, you used port `9999`. However, for this phase, you must connect to port `9997` instead.

This change is required because several API endpoints used in earlier phases (e.g., `ReqMsg`, `SendMsg`, and `ReqKeyBundle`) have been internally modified to support the PQXDH operations. While the endpoint names remain the same, their implementations differ between phases and therefore run on different server ports.

If you wish to test or demonstrate your implementations for Phases I and II, you may continue to use port `9999`, which still supports the original X3DH-based functionality. However, this port does not support the PQXDH or conference keying extensions introduced in Phase III. For Phase III, always use port `9997`.

Note that the server running on port `9997` does not retain any `IK`s, `SPK`s, or `OTK`s generated in previous phases. As a result, all required keys must be generated and registered again for Phase III.

4.1 Conference Keying

In this part, you are expected to generate a conference key with 3 other users for group communication. The other 3 users are pseudo-clients with whom you will be able to exchange values via the server. In the end, all 4 users (you and the other 3 pseudo-clients) should agree on the same conference key, which would be used for the communication between the group participants. However, for this part, you are not required to implement the communication protocol using the generated conference key. You are only expected to show that you can agree on a conference key with the given pseudo-clients.

In the conference setting, you are referred to as U_1, while the other group members are labeled U_2, U_3, and U_4, respectively. You will use the conference keying algorithm described in the lecture slides titled "Chapter 11 - Key Establishment Protocols" on SUCourse, with one modification. In this version, the conference keying protocol will be reformulated in the context of Elliptic Curve cryptography. Your task is to transform the conference keying algorithm from the slides to the

Elliptic Curve setting (using the `Ed25519` curve, as you did in previous phases), so that you can compute the value of x_1 as well as the conference key itself.

Initially, each group member generates a private and public partial key pair, r_i and z_i, using the key generation algorithm given in Section 2.4. You must sign your public partial key z_1 using your identity key, $\mathtt{IK}_A$. The signature, for which a scheme is given in Section 2.4, must be generated for the concatenated form of the public partial key: $(\mathtt{z}_1\mathtt{.x} \,\|\, \mathtt{z}_1\mathtt{.y})$. Finally, you must send your signed partial key to the server in the form given below using the `ExchangePartialKeys` service shared with you in `Client_Phase3_Basics.py`.

```
{'ID': stuID , 'z1.X': z1.x, 'z1.Y': z1.y, 'R.X': R.x, 'R.Y': R.y, 'S': s},
```

This function call will also return you the public partial keys of the other users, specifically of the users U_2 and U_4.

After receiving these values, you are expected to calculate and broadcast x_1 using the `ExchangeXs` service call in the `Client_Phase3_Basics.py`. Again, you should sign the concatenated form of x_1 $(\mathtt{x}_1\mathtt{.x} \,\|\, \mathtt{x}_1\mathtt{.y})$ using your identity key, $\mathtt{IK}_A$, and send your x_1 value in the form of:

```
{'ID': stuID , 'x1.X': x1.x, 'x1.Y': x1.y, 'R.X': R.x, 'R.Y': R.y, 'S': s},
```

Once again, this function call will also return you the x_i values of the other participants.

Finally, you are expected to generate the conference key and check if your key is correct using the `BonusChecker` service.

4.2 End-to-End Secure Communication using PQXDH

In this part of Phase III, you are required to build the same end-to-end communication protocol you developed in Phase II, but this time using Post-Quantum Extended Diffie–Hellman (PQXDH) instead of X3DH.

PQXDH is a Kyber-based post-quantum cryptography upgrade to the Diffie–Hellman key exchange, and it is currently being used in the original Signal Protocol.

In contrast to elliptic-curve Diffie-Hellman, post-quantum key agreement in PQXDH is performed using a key encapsulation mechanism (KEM). In this phase, you will use Kyber via the `kyber-py` library, which you can find at `https://github.com/GiacomoPope/kyber-py`. You will use the following functions for key generation and utilization:

- `keygen()` generates a public-private key pair, `(pk, sk)`.
- `encaps(pk)` produces a shared secret key `SS` and a ciphertext `c`.
- `decaps(sk, c)` reconstructs the same shared secret key `SS`.

The sender keeps the shared secret key `SS` and transmits only the ciphertext `c` to the receiver. The receiver uses its private key to recover the same shared secret.

4.2.1 Key Generation

In this phase, the mechanisms for generating IK, SPK, and OTK remain unchanged from the first phase of the project. However, a new type of keys, one-time post-quantum key encapsulation keys (PQOTK), needs to be generated to enable the PQXDH protocol.

The PQOTK keys are used alongside OTKs during the PQXDH protocol. As was in Phase I, we expect you to generate 10 OTKs, as well as 10 PQOTKs this time. Note that IK, SPK, and OTKs are Elliptic Curve (EC) keys, and PQOTK is a post-quantum key. For this reason, the generations of PQOTKs differ from the key generation algorithm you used for EC keys. To generate a public and private PQOTK pair, you are expected to use the default key generation function provided by the kyber-py library. For ease of representation, we will refer to PQOTK's public part as PQPK, and private part as PQSK throughout this documentation.

After generating your PQOTK, you are expected to sign the public part of your PQOTK, PQPK, using your IK, and send it to the server using the PQOTKReg API endpoint. Your PQPK should be represented in hexadecimal format. The message you send to the server should be in the following format:

```
{'ID': stuID, 'PQOTKID': pqotkID, 'PQPK': PQPK, 'R.X': R.x, 'R.Y': R.y, 'S': s}
```

where PQOTKID denotes the ID of your PQOTK. As was the case for your OTKs, you must start generating your PQOTK from 0 and follow the order.

4.2.2 Receiving Messages

The receiving messages portion of this phase is identical to the portion in Phase II. As you did in Phase II, you will call the PseudoSendMsg endpoint to receive 5 messages from the pseudo-client 18007. However, when you request messages from your inbox, they will be returned in the following format:

```
{'IDB': stuIDB, 'OTKID': otkID, 'MSGID': msgID, 'MSG': msg , 'IK.X': IK.x ,'IK.Y':
IK.y, 'EK.X': EK.x ,'EK.Y': EK.y, 'PQKEYID': pqkeyID, 'PQCT': PQCT },
```

where PQKEYID denotes the ID of the receiver's PQOTK used by the sender, and PQCT is the Kyber ciphertext produced during encapsulation using that PQOTK. Note that PQCT will be given in hexadecimal format.

After you receive a message, you can move on to establish the session key K_S using PQXDH. This portion is again mostly similar to the portion given in Section 3.1.1. The calculation for T1, T2, T3, and T4, remains identical. However, we now have to incorporate the post-quantum shared secret key SS into K_S generation. To do so, you first have to recover the shared secret key SS by decapsulating the PQCT you received from the pseudoclient. Once you achieve this, you will generate K_S as follows:

- $\mathtt{U} = \{\mathtt{T1.x} \parallel \mathtt{T1.y} \parallel \mathtt{T2.x} \parallel \mathtt{T2.y} \parallel \mathtt{T3.x} \parallel \mathtt{T3.y} \parallel \mathtt{T4.x} \parallel \mathtt{T4.y} \parallel \mathtt{SS} \parallel \mathtt{b'WhatsUpDoc'}\}$
- $K_S = \mathtt{SHA3_256}(\mathtt{U})$

Once K_S is generated, the same key derivation function (KDF) chain as given in Section 3.1.2 is used to generate the encryption key, HMAC key, and the KDF key for the next iteration.

As was in Phase II, you will receive a message with an incorrect HMAC, and several messages can be deleted. You are expected to display the final message block the same way as you did in Phase II.

4.2.3 Sending Messages

This part will also follow the same structure as in Phase II. Once you receive the 5 messages from the pseudo-client, you should encrypt the undeleted messages you received with valid MAC values. Finally, you will send the encrypted messages back to the pseudo-client (i.e., ID: `18007`) in the same order.

In this portion, the format of the receiver's pre-key bundle obtained from the server is changed to include PQ key information. The format is given below:

```
{'OTKID': otkID, 'IK.X': IK.x, 'IK.Y': IK.y, 'SPK.X': SPK.x, 'SPK.Y': SPK.y, 'SPK.R.X':
SPK.R.x, 'SPK.R.Y': SPK.R.y, 'SPK.S': SPK.s, 'OTK.X': OTK.x, 'OTK.Y': OTK.y, 'PQKEYID':
pqkeyID, 'PQPK': PQPK, 'PQPK.R.X': PQPK.R.x, 'PQPK.R.Y': PQPK.R.y 'PQPK.S': PQPK.S}
```

where `PQKEYID` identifies which (`PQOTK`) of the receiver is being used, and `PQPK` is the corresponding post-quantum public key, which the sender uses to derive a shared secret `SS` via Kyber. The fields `PQPK.R.X`, `PQPK.R.Y`, and `PQPK.S` contain the signature on `PQPK`, generated using the receiver's identity key. Note that `PQPK` is given in hexadecimal format.

You must verify the signatures for `SPK` and `PQPK` before using these keys.

After you successfully received the key bundle and completed the verification of the signatures, you should generate the ephemeral key pair `EK` and compute the K_S.

Finally, you should send your message to the server in the form of:

```
{'IDA': stuIDA, 'IDB': stuIDB, 'OTKID': otkID, 'MSGID': msgID, 'MSG': msg, 'IK.X':
IK.x ,'IK.Y': IK.y, 'EK.X': EK.x ,'EK.Y': EK.y, 'PQOTKID': pqotkID, 'PQCT': PQCT}
```

where the newly introduced two fields, PQOTKID and `PQCT`, denote the PQOTK ID of the receiver and the post-quantum ciphertext generated by the sender using the receiver's post-quantum public key `PQPK`, respectively. Note that `PQCT` should be given in hexadecimal format.

In this phase, the `Status` API endpoint that you used in Phase II to monitor your `OTK`s and inbox has been modified to include the number of `PQOTK`s remaining. You can use this function to monitor both your `PQOTK`s and `OTK`s, as well as your inbox, and generate new keys if needed.

5 Appendix I: Timeline, Deliverables, Weight & Policies

Project Phases	Deliverables	Due Date	Weight
Project announcement		14/12/2025	
First Phase	File: Client_Phase1.py	21/12/2025	30%
Second Phase	File: Client_Phase2.py	28/12/2025	40%
Third Phase	File: Client_Phase3.py	04/01/2025	30%

5.1 Policies

- The URL that you will send queries to for Phases III is: `http://harpoon1.sabanciuniv.edu:9997/`
- The URL that you will send queries to for Phases I and II is: `http://harpoon1.sabanciuniv.edu:9999/`
- You may work in groups of two.
- Submit all deliverables in the zip file "cs411_507_tp3_surname1_surname2.zip".
- Only one submission from one student is enough. However, the 2nd policy point should be respected.
- For codes, only .py scripts should be uploaded.
- **Points will be deducted if you don't respect the naming and file format criteria.**
- You may be asked to demonstrate the project phases to a TA or the instructor. During the demo, you might be asked to showcase the specified functionalities (e.g., registering keys then deleting keys, etc.). To avoid any issues, it is crucial to ensure that the final version of your code is designed to handle interactions dynamically and does not rely on fixed/hard-coded values for its functionality. You should demonstrate your code in a timely and efficient manner to avoid losing points, so please be aware of this early on in the implementation process.
- Since you will interact with a remote server to register your keys, send and receive messages. This server script itself is a validation method to check whether your codes are working correctly or not. We will also use it to check your implementation. If your implementation in a project phase fails to pass the validation, you will get no credit for that phase.
- Your codes will be checked for their similarity to other students' codes; and if the similarity score exceeds a certain threshold you will not get any credit for your work.